

LAPORAN PRAKTIKUM 6
STRUKTUR DATA
IMPLEMENTASI *ABSTRACT DATA TYPE (ADT)*
DOUBLE LINKEDLIST* MENGGUNAKAN *JAVA



Disusun Oleh:

Naysha Aprilia Rizki

2511531014

Dosen Pengampu:

Wahyudi. Dr. S.T.M.T

Asisten Praktikum:

Muhammad Galid Avero

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum yang berjudul “Laporan Praktikum 6 Struktur Data: Implementasi Abstract Data Type (ADT) Double LinkedList Menggunakan Bahasa Java” ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum pada mata kuliah Struktur Data. Adapun tujuan dari praktikum ini adalah untuk memahami konsep Abstract Data Type (ADT), khususnya pada struktur data **Double Linked List**, serta mengimplementasikannya menggunakan bahasa pemrograman Java. Selain itu, praktikum ini juga bertujuan agar mahasiswa mampu memahami cara kerja node yang saling terhubung dua arah melalui pointer next dan prev, sehingga proses penambahan (insert), penghapusan (delete), pencarian (search), serta traversal data dapat dilakukan dari arah depan maupun belakang secara efisien.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai struktur data, khususnya penerapan **Double Linked List** dalam pemrograman Java.

Padang, 13 Mei 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam ilmu komputer yang digunakan untuk mengatur dan menyimpan data agar dapat diproses secara efisien. Salah satu struktur data yang sering digunakan adalah Double Linked List. Double Linked List merupakan struktur data linear yang terdiri dari sekumpulan node, di mana setiap node berisi data serta dua buah pointer (referensi), yaitu pointer next yang menunjuk ke node berikutnya dan pointer prev yang menunjuk ke node sebelumnya.

Berbeda dengan array yang memiliki ukuran tetap dan penyimpanan berurutan dalam memori, Double Linked List bersifat dinamis sehingga ukuran data dapat bertambah atau berkurang sesuai kebutuhan. Struktur data ini memberikan fleksibilitas lebih dalam pengelolaan data karena traversal dapat dilakukan dari dua arah, yaitu dari depan ke belakang maupun dari belakang ke depan.

Dalam pemrograman, Double Linked List banyak digunakan untuk memanipulasi data secara dinamis, seperti proses penambahan (insert), penghapusan (delete), dan pencarian (search) data dengan lebih efisien dibandingkan struktur linear satu arah. Implementasi struktur data ini dalam bahasa pemrograman Java dilakukan dengan menggunakan konsep objek dan referensi, sehingga mahasiswa dapat memahami cara kerja pointer secara lebih nyata. Oleh karena itu, praktikum ini dilakukan agar mahasiswa dapat memahami konsep Double Linked List serta mampu mengimplementasikannya dalam bahasa pemrograman Java.

1.2 Tujuan

1. Memahami konsep dasar struktur data Double LinkedList
2. Mengimplementasikan Double LinkedList dalam Bahasa pemrograman Java.
3. Memahami Operasi dasar pada Double LinkedList seperti penambahan, penghapusan dan pencarian data.
4. Melatih kemampuan pemrograman dalam mengelola data secara dinamis menggunakan struktur data.

1.3 Manfaat

1. Menambah pemahaman tentang struktur data Double LinkedList.
2. Menjadi dasar pembelajaran untuk struktur data lain yang lebih kompleks.
3. Membantu memahami pengelolaan data secara dinamis dalam program.

BAB II

PEMBAHASAN

Dalam pemrograman, struktur data merupakan cara untuk menyimpan dan mengorganisasi data agar dapat digunakan secara efisien. Salah satu konsep penting dalam struktur data adalah Abstract Data Type (ADT). ADT adalah suatu tipe data abstrak yang mendefinisikan sekumpulan nilai dan operasi yang dapat dilakukan terhadap data tersebut tanpa menjelaskan bagaimana implementasinya secara detail.

Konsep utama dari ADT adalah pemisahan antara interface (apa yang dapat dilakukan) dan implementasi (bagaimana cara melakukannya). Dengan menggunakan ADT, programmer dapat lebih fokus pada fungsi dan kegunaan suatu tipe data tanpa harus memahami detail internalnya. Hal ini membuat program menjadi lebih modular, mudah dipelihara, dan mudah dikembangkan.

Dalam bahasa pemrograman Java, ADT biasanya direpresentasikan menggunakan class. Sebuah class berisi atribut (data) dan metode (fungsi) yang mendefinisikan perilaku dari objek tersebut. Atribut digunakan untuk menyimpan data, sedangkan metode digunakan untuk melakukan operasi terhadap data tersebut.

Pada praktikum ini, ADT yang diimplementasikan adalah ADT Double Linked List. Double Linked List merupakan struktur data linear yang terdiri dari sekumpulan node, di mana setiap node memiliki tiga bagian, yaitu pointer prev yang menunjuk ke node sebelumnya, data, dan pointer next yang menunjuk ke node berikutnya. Struktur ini bersifat dinamis karena ukuran data dapat berubah sesuai kebutuhan, berbeda dengan array yang memiliki ukuran tetap.

Double Linked List digunakan untuk mengelola data secara berurutan, di mana setiap elemen saling terhubung melalui pointer dua arah. Akses data dapat dilakukan dari node pertama (head) menuju node terakhir maupun sebaliknya dari node terakhir menuju node pertama. Kemampuan traversal dua arah ini menjadi salah satu keunggulan utama Double Linked List dibandingkan Single Linked List.

Selain itu, ADT Double Linked List juga dilengkapi dengan beberapa operasi, antara lain:

1. Penambahan data di depan (insert at front), yaitu menambahkan node baru di awal list.
2. Penambahan data di belakang (insert at end), yaitu menambahkan node baru di akhir list.

3. Penambahan data pada posisi tertentu (insert at position), yaitu menyisipkan node pada posisi tertentu dalam list.
4. Penghapusan data (delete), yaitu menghapus node baik di awal, akhir, maupun pada posisi tertentu.
5. Pencarian data (search), yaitu mencari apakah suatu data terdapat dalam list.
6. Penelusuran data (traversal), yaitu menampilkan seluruh isi linked list secara berurutan dari depan maupun belakang.

Dalam implementasinya, Double Linked List menggunakan konsep referensi (pointer) antar node. Setiap node akan menunjuk ke node berikutnya dan node sebelumnya sehingga membentuk hubungan dua arah. Keunggulan dari struktur ini adalah fleksibilitas dalam penambahan dan penghapusan data serta kemudahan traversal dua arah tanpa perlu menggeser elemen seperti pada array. Namun, kelemahannya adalah membutuhkan memori lebih banyak karena setiap node memiliki dua pointer.

Dengan memahami dan mengimplementasikan ADT Double Linked List dalam bahasa Java, mahasiswa dapat mengembangkan kemampuan dalam mengelola data secara dinamis serta meningkatkan pemahaman terhadap konsep dasar struktur data dalam pemrograman.

1. Langkah Kerja

1.1. NodeDLL_2511531014

- Kode Program

```
1 package pekan6_2511531014;
2
3 public class NodeDLL_2511531014 {
4     // mendefinisikan kelas node
5     int data_1014; //data
6     NodeDLL_2511531014 next_1014; // Pointer ke next node
7     NodeDLL_2511531014 prev_1014; // Pointer ke previous node
8
9     //konstruktor
10    public NodeDLL_2511531014 (int data_1014) {
11        this.data_1014= data_1014;
12        this.next_1014 = next_1014;
13        this.prev_1014 = prev_1014;
14    }
15 }
16 }
```

- Pembahasan

Program ini dibuat dengan mendefinisikan class NodeDLL_2511531014 yang digunakan sebagai representasi node pada struktur data Double

Linked List (DLL). Class ini berfungsi sebagai elemen dasar yang menyusun linked list.

1. Class ini memiliki atribut `int data_1014` yang digunakan untuk menyimpan nilai data pada node, serta atribut `NodeDLL_2511531014 next_1014` dan `NodeDLL_2511531014 prev_1014` yang berfungsi sebagai pointer atau referensi ke node berikutnya dan node sebelumnya dalam linked list.

```
1 package pekan6_2511531014;
2
3 public class NodeDLL_2511531014 {
4     // mendefinisikann kelas node
5     int data_1014; //data
6     NodeDLL_2511531014 next_1014; // Pointer ke next node
7     NodeDLL_2511531014 prev_1014; // Pointer ke previous node
8 }
```

2. Konstruktor `NodeDLL_2511531014(int data_1014)` digunakan untuk menginisialisasi node dengan nilai tertentu. Pada konstruktor ini, atribut `data_1014` diisi dengan nilai yang diberikan. Penggunaan keyword `this` digunakan untuk membedakan antara atribut class dan parameter yang memiliki nama yang sama, sehingga nilai dapat disimpan dengan benar ke dalam atribut.

```
9 //konstruktor
10 public NodeDLL_2511531014 (int data_1014) {
11     this.data_1014= data_1014;
12     this.next_1014 = next_1014;
13     this.prev_1014 = prev_1014;
14
15 }
16 }
```

3. Secara keseluruhan, class ini digunakan sebagai komponen utama dalam pembentukan struktur data Double Linked List, di mana setiap node menyimpan data serta memiliki dua referensi, yaitu ke node berikutnya dan node sebelumnya, sehingga memungkinkan traversal data dilakukan secara maju maupun mundur.

1.2. InsertDLL_2511531014

- Kode Program

```

1 package pekan6_2511531014;
2
3 public class InsertDLL_2511531014 {
4
5     // menambahkan node pada awal DLL
6     static NodeDLL_2511531014 insertBegin(NodeDLL_2511531014 head_1014, int data_1014) {
7         // buat node baru
8         NodeDLL_2511531014 new_node = new NodeDLL_2511531014(data_1014);
9         // jadikan pointer next nya head
10        new_node.next_1014 = head_1014;
11        // jadikan pointer prev head ke new node
12        if (head_1014 != null) {
13            head_1014.prev_1014 = new_node;
14        }
15        return new_node;
16    }
17
18    // fungsi menambah node di akhir
19    public static NodeDLL_2511531014 insertEnd(NodeDLL_2511531014 head_1014, int newData_1014) {
20        // buat node baru
21        NodeDLL_2511531014 newNode = new NodeDLL_2511531014(newData_1014);
22        // jika DLL kosong
23        if (head_1014 == null) {
24            head_1014 = newNode;
25        }
26        else {
27            NodeDLL_2511531014 curr_1014 = head_1014;
28            while (curr_1014.next_1014 != null) {
29                curr_1014 = curr_1014.next_1014;
30            }
31            curr_1014.next_1014 = newNode;
32            newNode.prev_1014 = curr_1014;
33        }
34        return head_1014;
35    }
36 }

```

```

37
38 // fungsi menambahkan node ke posisi tertentu
39 public static NodeDLL_2511531014 insertAtPosition(NodeDLL_2511531014 head_1014, int pos,
40     int new_data) {
41     // buat node baru
42     NodeDLL_2511531014 new_node = new NodeDLL_2511531014(new_data);
43     if (pos == 1) {
44         new_node.next_1014 = head_1014;
45         if (head_1014 != null) {
46             head_1014.prev_1014 = new_node;
47         }
48         head_1014 = new_node;
49         return head_1014;
50     }
51
52     NodeDLL_2511531014 curr_1014 = head_1014;
53     for (int i = 1; i < pos - 1 && curr_1014 != null; ++i) {
54         curr_1014 = curr_1014.next_1014;
55     }
56     if (curr_1014 == null) {
57         System.out.println("Posisi tidak ada");
58         return head_1014;
59     }
60
61     new_node.prev_1014 = curr_1014;
62     new_node.next_1014 = curr_1014.next_1014;
63     curr_1014.next_1014 = new_node;
64     if (new_node.next_1014 != null) {
65         new_node.next_1014.prev_1014 = new_node;
66     }
67     return head_1014;
68 }
69
70 public static void printList(NodeDLL_2511531014 head_1014) {
71     NodeDLL_2511531014 curr_1014 = head_1014;
72     while (curr_1014 != null) {
73         System.out.print(curr_1014.data_1014 + " <-> ");
74         curr_1014 = curr_1014.next_1014;
75     }
76     System.out.println();
77 }
78 }

```

```

public static void main(String[] args) {
    // membuat dll 2 <-> 3 <-> 5
    NodeDLL_2511531014 head_1014 = new NodeDLL_2511531014(2);
    head_1014.next_1014 = new NodeDLL_2511531014(3);
    head_1014.next_1014.prev_1014 = head_1014;
    head_1014.next_1014.next_1014 = new NodeDLL_2511531014(5);
    head_1014.next_1014.next_1014.prev_1014 = head_1014.next_1014;

    // cetak DLL awal
    System.out.print("DLL Awal 2511531014 : ");
    printList(head_1014);

    // tambah 1 di awal
    head_1014 = insertBegin(head_1014, 1);
    System.out.print(
        "Simpul 1 ditambah di awal 2511531014 : ");
    printList(head_1014);

    // tambah 6 di akhir
    System.out.print(
        "Simpul 6 ditambah di akhir 2511531014 : ");
    int data_1014 = 6;
    head_1014 = insertEnd(head_1014, data_1014);
    printList(head_1014);

    // menambahkan node 4 di posisi 4
    System.out.print("Tambah node 4 di posisi 4 2511531014 : ");
    int data2_1014 = 4;
    int pos_1014 = 4;
    head_1014 = insertAtPosition(head_1014, pos_1014, data2_1014);
    printList(head_1014);
}
}

```

• Pembahasan

Program ini dibuat dengan mendefinisikan class InsertDLL_2511531014 yang digunakan untuk melakukan operasi penambahan node pada struktur data Double Linked List (DLL). Class ini berfungsi untuk mengelola proses insert node di awal, akhir, maupun posisi tertentu pada linked list.

1. Method insertBegin() digunakan untuk menambahkan node di awal Double Linked List. Program membuat node baru, kemudian pointer next dari node baru diarahkan ke head lama. Jika linked list tidak kosong, maka pointer prev dari head lama diarahkan ke node baru sehingga node baru menjadi head pertama.

```

1 package pekan6_2511531014;
2
3 public class InsertDLL_2511531014 {
4
5     // menambahkan node pada awal DLL
6     static NodeDLL_2511531014 insertBegin(NodeDLL_2511531014 head_1014, int data_1014) {
7         // buat node baru
8         NodeDLL_2511531014 new_node = new NodeDLL_2511531014(data_1014);
9         // jadikan pointer next nya head
10        new_node.next_1014 = head_1014;
11        // jadikan pointer prev head ke new node
12        if (head_1014 != null) {
13            head_1014.prev_1014 = new_node;
14        }
15        return new_node;
16    }
17 }

```

2. Method insertEnd() digunakan untuk menambahkan node di akhir Double Linked List. Program akan membuat node baru lalu melakukan traversal hingga node terakhir ditemukan. Setelah itu, pointer next node terakhir diarahkan ke node baru dan pointer prev node baru diarahkan ke node sebelumnya. Jika linked list kosong, node baru langsung menjadi head.

```

}

// fungsi menambah node di akhir
public static NodeDLL_2511531014 insertEnd(NodeDLL_2511531014 head_1014, int newData_1014) {
    // buat node baru
    NodeDLL_2511531014 newNode = new NodeDLL_2511531014(newData_1014);
    // jika DLL kosong
    if (head_1014 == null) {
        head_1014 = newNode;
    }
    else {
        NodeDLL_2511531014 curr_1014 = head_1014;
        while (curr_1014.next_1014 != null) {
            curr_1014 = curr_1014.next_1014;
        }
        curr_1014.next_1014 = newNode;
        newNode.prev_1014 = curr_1014;
    }
    return head_1014;
}

```

3. Method `insertAtPosition()` digunakan untuk menambahkan node pada posisi tertentu dalam linked list. Program melakukan traversal sampai posisi yang diinginkan, kemudian pointer `next` dan `prev` diatur agar node baru terhubung dengan node sebelum dan sesudahnya. Jika posisi tidak ditemukan, program menampilkan pesan “Posisi tidak ada”.

```

// fungsi menambahkan node ke posisi tertentu
public static NodeDLL_2511531014 insertAtPosition(NodeDLL_2511531014 head_1014, int pos,
int new_data) {
    // buat node baru
    NodeDLL_2511531014 new_node = new NodeDLL_2511531014(new_data);
    if (pos == 1) {
        new_node.next_1014 = head_1014;
        if (head_1014 != null) {
            head_1014.prev_1014 = new_node;
        }
        head_1014 = new_node;
        return head_1014;
    }

    NodeDLL_2511531014 curr_1014 = head_1014;
    for (int i = 1; i < pos - 1 && curr_1014 != null; ++i) {
        curr_1014 = curr_1014.next_1014;
    }
    if (curr_1014 == null) {
        System.out.println("Posisi tidak ada");
        return head_1014;
    }

    new_node.prev_1014 = curr_1014;
    new_node.next_1014 = curr_1014.next_1014;
    curr_1014.next_1014 = new_node;
    if (new_node.next_1014 != null) {
        new_node.next_1014.prev_1014 = new_node;
    }
    return head_1014;
}

```

4. Method `printList()` digunakan untuk menampilkan seluruh isi Double Linked List secara berurutan dari depan ke belakang menggunakan traversal pointer `next`.

```

}

public static void printList(NodeDLL_2511531014 head_1014) {
    NodeDLL_2511531014 curr_1014 = head_1014;
    while (curr_1014 != null) {
        System.out.print(curr_1014.data_1014 + " <-> ");
        curr_1014 = curr_1014.next_1014;
    }
    System.out.println();
}

```

5. Pada method `main()`, program membuat Double Linked List awal dengan data 2 <-> 3 <-> 5. Selanjutnya program melakukan beberapa operasi, yaitu menambahkan node 1 di awal list, menambahkan node 6 di akhir list, dan menambahkan node 4 pada posisi ke-4. Hasil setiap operasi kemudian ditampilkan menggunakan method `printList()`.

```

public static void main(String[] args) {
    // membuat dll 2 <-> 3 <-> 5
    NodeDLL_2511531014 head_1014 = new NodeDLL_2511531014(2);
    head_1014.next_1014 = new NodeDLL_2511531014(3);
    head_1014.next_1014.prev_1014 = head_1014;
    head_1014.next_1014.next_1014 = new NodeDLL_2511531014(5);
    head_1014.next_1014.next_1014.prev_1014 = head_1014.next_1014;

    // cetak DLL awal
    System.out.print("DLL Awal 2511531014 : ");
    printList(head_1014);

    // tambah 1 di awal
    head_1014 = insertBegin(head_1014, 1);
    System.out.print(
        "Simpul 1 ditambah di awal 2511531014 : ");
    printList(head_1014);

    // tambah 6 di akhir
    System.out.print(
        "Simpul 6 ditambah di akhir 2511531014 : ");
    int data_1014 = 6;
    head_1014 = insertEnd(head_1014, data_1014);
    printList(head_1014);

    // menambah node 4 di posisi 4
    System.out.print("Tambah node 4 di posisi 4 2511531014 : ");
    int data2_1014 = 4;
    int pos_1014 = 4;
    head_1014 = insertAtPosition(head_1014, pos_1014, data2_1014);
    printList(head_1014);
}
}

```

6. Dan berikut Adalah Output yang dihasilkan

```

<terminated> InsertDLL_2511531014 [Java Application] C:\Users\naysh\.p2\pool\p
DLL Awal 2511531014 : 2 <-> 3 <-> 5 <->
Simpul 1 ditambah di awal 2511531014 : 1 <-> 2 <-> 3 <-> 5 <->
Simpul 6 ditambah di akhir 2511531014 : 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
Tambah node 4 di posisi 4 2511531014 : 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

```

1.3. PenelusuranDLL_2511531014

- Kode Program

```

package pekan6_2511531014;

public class PenelusuranDLL_2511531014 {
    // fungsi penelusuran maju
    static void forwardTraversal(NodeDLL_2511531014 head_1014) {
        // memulai penelusuran dari head
        NodeDLL_2511531014 curr_1014 = head_1014;
        // lanjutkan sampai akhir
        while (curr_1014 != null) {
            // print data
            System.out.print(curr_1014.data_1014 + " <-> ");
            // pindah ke node berikutnya
            curr_1014 = curr_1014.next_1014;
        }
        // print spasi
        System.out.println();
    }

    // fungsi penelusuran mundur
    static void backwardTraversal(NodeDLL_2511531014 tail_1014) {
        // mulai dari akhir
        NodeDLL_2511531014 curr_1014 = tail_1014;
        // lanjut sampai head
        while (curr_1014 != null) {
            // cetak data
            System.out.print(curr_1014.data_1014 + " <-> ");
            // pindah ke node sebelumnya
            curr_1014 = curr_1014.prev_1014;
        }
        // cetak spasi
        System.out.println();
    }
}

```

```

4 public static void main(String[] args) {
5     // buat DLL
6     NodeDLL_2511531014 head_1014 = new NodeDLL_2511531014(1);
7     NodeDLL_2511531014 second_1014 = new NodeDLL_2511531014(2);
8     NodeDLL_2511531014 third_1014 = new NodeDLL_2511531014(3);
9
10    head_1014.next_1014 = second_1014;
11    second_1014.prev_1014 = head_1014;
12
13    second_1014.next_1014 = third_1014;
14    third_1014.prev_1014 = second_1014;
15
16    System.out.println("Penelusuran Maju 2511531014 :");
17    forwardTraversal(head_1014);
18
19    System.out.println("Penelusuran Mundur 2511531014 :");
20    backwardTraversal(third_1014);
21 }
22 }

```

• Pembahasan

Program ini dibuat dengan mendefinisikan class PenelusuranDLL_2511531014 yang digunakan untuk melakukan proses penelusuran (traversal) pada struktur data Double Linked List (DLL). Program ini berfungsi untuk menampilkan isi linked list baik secara maju maupun mundur.

1. Method forwardTraversal() digunakan untuk melakukan penelusuran maju pada Double Linked List. Proses traversal dimulai dari node head, kemudian program menampilkan data setiap node dan berpindah ke node berikutnya menggunakan pointer next_1014 hingga mencapai node terakhir (null).

```

1 package pekan6_2511531014;
2
3 public class PenelusuranDLL_2511531014 {
4     // fungsi penelusuran maju
5     static void forwardTraversal(NodeDLL_2511531014 head_1014) {
6         // memulai penelusuran dari head
7         NodeDLL_2511531014 curr_1014 = head_1014;
8         // lanjutkan sampai akhir
9         while (curr_1014 != null) {
10            // print data
11            System.out.print(curr_1014.data_1014 + " <-> ");
12            // pindah ke node berikutnya
13            curr_1014 = curr_1014.next_1014;
14        }
15        // print spasi
16        System.out.println();
17    }
18 }

```

2. Method backwardTraversal() digunakan untuk melakukan penelusuran mundur pada Double Linked List. Proses traversal dimulai dari node terakhir (tail), kemudian program menampilkan data setiap node dan berpindah ke node sebelumnya menggunakan pointer prev_1014 hingga mencapai node pertama (null).

```

    }
    // fungsi penelusuran mundur
    static void backwardTraversal(NodeDLL_2511531014 tail_1014) {
        // mulai dari akhir
        NodeDLL_2511531014 curr_1014 = tail_1014;
        // lanjut sampai head
        while (curr_1014 != null) {
            // cetak data
            System.out.print(curr_1014.data_1014 + " <-> ");
            // pindah ke node sebelumnya
            curr_1014 = curr_1014.prev_1014;
        }
        // cetak spasi
        System.out.println();
    }
}

```

3. Pada method main(), program membuat Double Linked List dengan data 1 <-> 2 <-> 3. Setiap node saling dihubungkan menggunakan pointer next dan prev. Setelah itu, program menampilkan hasil penelusuran maju menggunakan method forwardTraversal() dan penelusuran mundur menggunakan method backwardTraversal().

```

32
33 public static void main(String[] args) {
34
35     // cetak DLL
36     NodeDLL_2511531014 head_1014 = new NodeDLL_2511531014(1);
37     NodeDLL_2511531014 second_1014 = new NodeDLL_2511531014(2);
38     NodeDLL_2511531014 third_1014 = new NodeDLL_2511531014(3);
39
40     head_1014.next_1014 = second_1014;
41     second_1014.prev_1014 = head_1014;
42
43     second_1014.next_1014 = third_1014;
44     third_1014.prev_1014 = second_1014;
45
46     System.out.println("Penelusuran Maju 2511531014 :");
47     forwardTraversal(head_1014);
48
49     System.out.println("Penelusuran Mundur 2511531014 :");
50     backwardTraversal(third_1014);
51 }
52 }
53

```

4. Secara keseluruhan, program ini menunjukkan cara kerja traversal pada Double Linked List yang dapat dilakukan dalam dua arah, yaitu dari depan ke belakang maupun dari belakang ke depan, sehingga mempermudah proses pengelolaan data secara dinamis. Berikut Adalah Output hasilnya.

```

<terminated> PenelusuranDLL_2511531014 [Java Ap
Penelusuran Maju 2511531014 :
1 <-> 2 <-> 3 <->
Penelusuran Mundur 2511531014 :
3 <-> 2 <-> 1 <->

```

1.4. HapusDLL_2511531014

- Kode Program

```
package pekan6_2511531014;

public class HapusDLL_2511531014 {
    // fungsi menghapus node awal
    public static NodeDLL_2511531014 delHead_1014(NodeDLL_2511531014 head_1014) {
        if (head_1014 == null) {
            return null;
        }
        NodeDLL_2511531014 temp_1014 = head_1014;
        head_1014 = head_1014.next_1014;
        if (head_1014 != null) {
            head_1014.prev_1014 = null;
        }
        return head_1014;
    }

    // fungsi menghapus di akhir
    public static NodeDLL_2511531014 dellast_1014(NodeDLL_2511531014 head_1014) {
        if (head_1014 == null) {
            return null;
        }
        if (head_1014.next_1014 == null) {
            return null;
        }
        NodeDLL_2511531014 curr_1014 = head_1014;
        while (curr_1014.next_1014 != null) {
            curr_1014 = curr_1014.next_1014;
        }
        // update pointer previous node
        if (curr_1014.prev_1014 != null) {
            curr_1014.prev_1014.next_1014 = null;
        }
        return head_1014;
    }

    // fungsi menghapus node pada posisi tertentu
    public static NodeDLL_2511531014 delPos_1014(NodeDLL_2511531014 head_1014, int pos_1014) {
        // jika DLL kosong
        if (head_1014 == null) {
            return head_1014;
        }
        NodeDLL_2511531014 curr_1014 = head_1014;
        // hitung sampai ke node yang akan dihapus
        for (int i = 1; curr_1014 != null && i < pos_1014; ++i) {
            curr_1014 = curr_1014.next_1014;
        }
        // jika posisi tidak ditemukan
        if (curr_1014 == null) {
            return head_1014;
        }
        // update pointer
        if (curr_1014.prev_1014 != null) {
            curr_1014.prev_1014.next_1014 = curr_1014.next_1014;
        }
        if (curr_1014.next_1014 != null) {
            curr_1014.next_1014.prev_1014 = curr_1014.prev_1014;
        }
        // jika yang dihapus head
        if (head_1014 == curr_1014) {
            head_1014 = curr_1014.next_1014;
        }
        return head_1014;
    }

    // fungsi mencetak DLL
    public static void printList(NodeDLL_2511531014 head_1014) {
        NodeDLL_2511531014 curr_1014 = head_1014;
        while (curr_1014 != null) {
            System.out.println(curr_1014.data_1014 + " <-> ");
            curr_1014 = curr_1014.next_1014;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // BUKAN SEHAB DLL
        NodeDLL_2511531014 head_1014 = new NodeDLL_2511531014(1);
        head_1014.next_1014 = new NodeDLL_2511531014(2);
        head_1014.next_1014.prev_1014 = head_1014;
        head_1014.next_1014.next_1014 = new NodeDLL_2511531014(3);
        head_1014.next_1014.next_1014.prev_1014 = head_1014.next_1014;
        head_1014.next_1014.next_1014.next_1014 = new NodeDLL_2511531014(4);
        head_1014.next_1014.next_1014.next_1014.prev_1014 = head_1014.next_1014.next_1014;
        head_1014.next_1014.next_1014.next_1014.next_1014 = new NodeDLL_2511531014(5);
        head_1014.next_1014.next_1014.next_1014.next_1014.prev_1014 = head_1014.next_1014.next_1014.next_1014;

        System.out.println("DLL Awal: ");
        printList(head_1014);

        System.out.println("Setelah head dihapus: ");
        head_1014 = delHead_1014(head_1014);
        printList(head_1014);

        System.out.println("Setelah node terakhir dihapus: ");
        head_1014 = dellast_1014(head_1014);
        printList(head_1014);

        System.out.println("menghapus node ke 2: ");
        head_1014 = delPos_1014(head_1014, 2);
        printList(head_1014);
    }
}
```

- Pembahasan

Program ini dibuat dengan mendefinisikan class HapusDLL_2511531014 yang digunakan untuk melakukan operasi penghapusan node pada struktur data Double Linked List (DLL). Program ini berfungsi untuk menghapus node di awal, di akhir, maupun pada posisi tertentu dalam linked list.

1. Method `delHead_1014()` digunakan untuk menghapus node pertama (head) pada Double Linked List. Program akan memindahkan head ke node berikutnya menggunakan pointer `next`, kemudian pointer `prev` dari head baru diubah menjadi `null` agar tidak lagi terhubung dengan node lama.

```
package pekan6_2511531014;

public class HapusDLL_2511531014 {
    // fungsi menghapus node awal
    public static NodeDLL_2511531014 delHead_1014(NodeDLL_2511531014 head_1014) {
        if (head_1014 == null) {
            return null;
        }
        NodeDLL_2511531014 temp_1014 = head_1014;
        head_1014 = head_1014.next_1014;
        if (head_1014 != null) {
            head_1014.prev_1014 = null;
        }
        return head_1014;
    }
}
```

2. Method `delLast_1014()` digunakan untuk menghapus node terakhir pada Double Linked List. Program melakukan traversal hingga node terakhir ditemukan, kemudian pointer `next` dari node sebelumnya diubah menjadi `null` sehingga node terakhir terputus dari linked list.

```
}
// fungsi menghapus di akhir
public static NodeDLL_2511531014 delLast_1014(NodeDLL_2511531014 head_1014) {
    if (head_1014 == null) {
        return null;
    }
    if (head_1014.next_1014 == null) {
        return null;
    }
    NodeDLL_2511531014 curr_1014 = head_1014;
    while (curr_1014.next_1014 != null) {
        curr_1014 = curr_1014.next_1014;
    }
    // update pointer previous node
    if (curr_1014.prev_1014 != null) {
        curr_1014.prev_1014.next_1014 = null;
    }
    return head_1014;
}
```

3. Method `delPos_1014()` digunakan untuk menghapus node pada posisi tertentu. Program melakukan traversal hingga posisi yang ditentukan, kemudian pointer `next` dan `prev` dari node sebelum dan sesudahnya diperbarui agar node yang dipilih terhapus dari linked list. Jika node yang dihapus adalah head, maka head akan dipindahkan ke node berikutnya.

```
// fungsi menghapus node pada posisi tertentu
public static NodeDLL_2511531014 delPos_1014(NodeDLL_2511531014 head_1014, int pos_1014) {
    // jika DLL kosong
    if (head_1014 == null) {
        return head_1014;
    }
    NodeDLL_2511531014 curr_1014 = head_1014;
    // telusuri sampai ke node yang akan dihapus
    for (int i = 1; curr_1014 != null && i < pos_1014; ++i) {
        curr_1014 = curr_1014.next_1014;
    }
    // jika posisi tidak ditemukan
    if (curr_1014 == null) {
        return head_1014;
    }
    // update pointer
    if (curr_1014.prev_1014 != null) {
        curr_1014.prev_1014.next_1014 = curr_1014.next_1014;
    }
    if (curr_1014.next_1014 != null) {
        curr_1014.next_1014.prev_1014 = curr_1014.prev_1014;
    }
    // jika yang dihapus head
    if (head_1014 == curr_1014) {
        head_1014 = curr_1014.next_1014;
    }
    return head_1014;
}
// fungsi menghapus DLL
```

4. Method `printList()` digunakan untuk menampilkan seluruh isi Double Linked List secara berurutan dari depan ke belakang menggunakan traversal pointer `next`.

```
// fungsi mencetak DLL
public static void printList(NodeDLL_2511531014 head_1014) {
    NodeDLL_2511531014 curr_1014 = head_1014;
    while (curr_1014 != null) {
        System.out.print(curr_1014.data_1014 + " <-> ");
        curr_1014 = curr_1014.next_1014;
    }
    System.out.println();
}
```

5. Pada method `main()`, program membuat Double Linked List dengan data 1 <-> 2 <-> 3 <-> 4 <-> 5. Setelah itu dilakukan beberapa operasi penghapusan, yaitu menghapus node pertama (head), menghapus node terakhir, dan menghapus node pada posisi ke-2. Hasil setiap operasi kemudian ditampilkan menggunakan method `printList()`.

```
public static void main(String[] args) {
    // ===== DLL
    NodeDLL_2511531014 head_1014 = new NodeDLL_2511531014(1);
    head_1014.next_1014 = new NodeDLL_2511531014(2);
    head_1014.next_1014.prev_1014 = head_1014;
    head_1014.next_1014.next_1014 = new NodeDLL_2511531014(3);
    head_1014.next_1014.next_1014.prev_1014 = head_1014.next_1014;
    head_1014.next_1014.next_1014.next_1014 = new NodeDLL_2511531014(4);
    head_1014.next_1014.next_1014.next_1014.prev_1014 = head_1014.next_1014.next_1014;
    head_1014.next_1014.next_1014.next_1014.next_1014 = new NodeDLL_2511531014(5);
    head_1014.next_1014.next_1014.next_1014.next_1014.prev_1014 = head_1014.next_1014.next_1014.next_1014;

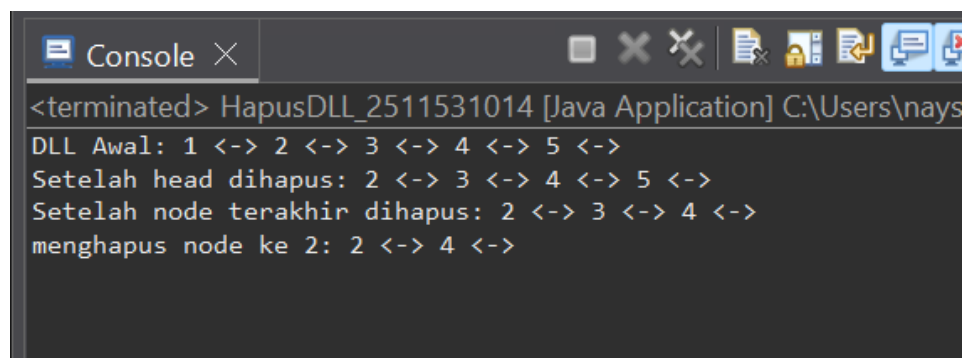
    System.out.print("DLL Awal: ");
    printList(head_1014);

    System.out.print("Setelah head dihapus: ");
    head_1014 = delHead_1014(head_1014);
    printList(head_1014);

    System.out.print("Setelah node terakhir dihapus: ");
    head_1014 = delLast_1014(head_1014);
    printList(head_1014);

    System.out.print("menghapus node ke 2: ");
    head_1014 = delPos_1014(head_1014, 2);
    printList(head_1014);
}
```

6. Berikut Adalah hasil outputnya



The screenshot shows a console window titled "Console" with the following output:

```
<terminated> HapusDLL_2511531014 [Java Application] C:\Users\nays
DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->
Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->
Setelah node terakhir dihapus: 2 <-> 3 <-> 4 <->
menghapus node ke 2: 2 <-> 4 <->
```

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi struktur data Double Linked List (DLL) telah berhasil dilakukan dengan baik serta mampu merepresentasikan konsep Abstract Data Type (ADT) berbasis pointer. Struktur data ini memungkinkan penyimpanan data secara dinamis, di mana setiap elemen (node) saling terhubung melalui dua referensi, yaitu ke node berikutnya (next) dan node sebelumnya (prev).

Melalui berbagai program yang telah dibuat, seperti pembuatan node, penambahan data (di awal, akhir, dan posisi tertentu), penelusuran maju dan mundur, pencarian data, serta penghapusan node (di awal, akhir, dan posisi tertentu), dapat disimpulkan bahwa operasi pada Double Linked List dilakukan dengan cara menelusuri dan memanipulasi pointer antar node. Hal ini berbeda dengan struktur berbasis array yang menggunakan indeks.

Pada implementasi Double Linked List, kelebihan yang diperoleh adalah fleksibilitas dalam penambahan dan penghapusan data serta kemampuan traversal dua arah tanpa perlu menggeser elemen seperti pada array. Selain itu, proses penelusuran dapat dilakukan dari depan maupun belakang sehingga lebih efisien dibandingkan Single Linked List dalam beberapa kondisi. Namun, struktur ini membutuhkan penggunaan memori yang lebih besar karena setiap node memiliki dua pointer.

Dengan adanya berbagai operasi yang telah diuji, seperti traversal, insertion, deletion, dan searching, dapat disimpulkan bahwa struktur data Double Linked List sangat efektif digunakan dalam pengelolaan data yang bersifat dinamis dan sering mengalami perubahan.

Secara keseluruhan, praktikum ini memberikan pemahaman yang lebih mendalam mengenai konsep Double Linked List, cara kerja pointer next dan prev antar node, serta penerapannya dalam pemrograman Java, sehingga dapat meningkatkan kemampuan dalam merancang dan mengelola struktur data secara efisien dan terstruktur.

TUGAS PRAKTIKUM PEKAN 6

Topik : Implementasi Double Linked List

Tema : Playlist Lagu Sederhana

Nama Class “Lagu_2511531014”

- **Kode Program**

```
1 package TugasPekan6_2511531014;
2
3 public class Lagu_2511531014 {
4
5     // mendefinisikan class node lagu
6     String judul_1014;
7     String penyanyi_1014;
8
9     // pointer ke next dan previous node
10    Lagu_2511531014 next_1014; //pointer untuk maju
11    Lagu_2511531014 prev_1014; //pointer untuk mundur
12
13    // konstruktor
14    public Lagu_2511531014(String judul_1014, String penyanyi_1014) {
15        this.judul_1014 = judul_1014;
16        this.penyanyi_1014 = penyanyi_1014;
17        this.next_1014 = null;
18        this.prev_1014 = null;
19    }
20 }
```

- **Pembahasan**

Program ini dibuat dengan mendefinisikan class Lagu_2511531014 yang digunakan sebagai representasi node pada struktur data Double Linked List (DLL). Class ini berfungsi sebagai elemen dasar yang menyusun playlist lagu.

1. Class ini memiliki atribut judul_1014 dan penyanyi_1014 yang bertipe String untuk menyimpan informasi judul lagu dan nama penyanyi. Selain itu, terdapat atribut next_1014 dan prev_1014 yang berfungsi sebagai pointer atau referensi ke node berikutnya dan node sebelumnya dalam Double Linked List.

```
package TugasPekan6_2511531014;

public class Lagu_2511531014 {

    // mendefinisikan class node lagu
    String judul_1014;
    String penyanyi_1014;
```

2. Pointer next_1014 digunakan untuk menghubungkan node ke lagu berikutnya (maju), sedangkan pointer prev_1014 digunakan untuk

menghubungkan node ke lagu sebelumnya (mundur). Dengan adanya dua pointer ini, traversal data dapat dilakukan dari dua arah.

```
// pointer ke next dan previous node
Lagu_2511531014 next_1014; //pointer untuk maju
Lagu_2511531014 prev_1014; //pointer untuk mundur
```

3. Konstruktor `Lagu_2511531014(String judul_1014, String penyanyi_1014)` digunakan untuk menginisialisasi node lagu baru. Pada konstruktor ini, atribut `judul_1014` dan `penyanyi_1014` diisi sesuai nilai yang diberikan, sedangkan pointer `next_1014` dan `prev_1014` diatur bernilai `null` yang menandakan bahwa node belum terhubung dengan node lain.

```
// konstruktor
public Lagu_2511531014(String judul_1014, String penyanyi_1014) {
    this.judul_1014 = judul_1014;
    this.penyanyi_1014 = penyanyi_1014;
    this.next_1014 = null;
    this.prev_1014 = null;
}
}
```

4. Secara keseluruhan, class ini digunakan sebagai komponen utama dalam pembentukan struktur data Double Linked List pada playlist musik, di mana setiap node menyimpan data lagu dan memiliki hubungan dua arah dengan node lainnya melalui pointer `next` dan `prev`.

Nama Class “Musik_2511531014”

- **Kode Program**

```
1 package TugasPekan6_2511531014;
2 import java.util.Scanner;
3 public class Musik_2511531014 {
4
5     Lagu_2511531014 head_1014 = null;
6     Lagu_2511531014 tail_1014 = null;
7
8     // tambah lagu di akhir
9     public void tambahLagu_1014(String judul_1014, String penyanyi_1014) {
10         Lagu_2511531014 laguBaru_1014 =
11             new Lagu_2511531014(judul_1014, penyanyi_1014);
12
13         // jika playlist kosong
14         if (head_1014 == null) {
15             head_1014 = laguBaru_1014;
16             tail_1014 = laguBaru_1014;
17         } else {
18             tail_1014.next_1014 = laguBaru_1014;
19             laguBaru_1014.prev_1014 = tail_1014;
20             tail_1014 = laguBaru_1014;
21         }
22         System.out.println("Lagu berhasil ditambahkan!");
23     }
24
25     // hapus lagu pertama
26     public void hapusLaguAwal_1014() {
27         if (head_1014 == null) {
28             System.out.println("Playlist kosong!");
29             return;
30         }
31         System.out.println("Lagu "
32             + head_1014.judul_1014
33             + " berhasil dihapus");
34         head_1014 = head_1014.next_1014;
35         if (head_1014 != null) {
36             head_1014.prev_1014 = null;
37         } else {
38             tail_1014 = null;
39         }
40     }
41 }
```

```

10
11 // tampil playlist maju
12 public void tampilMaju_1014() {
13     if (head_1014 == null) {
14         System.out.println("Playlist kosong!");
15         return;
16     }
17     Lagu_2511531014 curr_1014 = head_1014;
18     System.out.println("\n=== Playlist Maju ===");
19     while (curr_1014 != null) {
20         System.out.println(
21             curr_1014.judul_1014
22             + " - "
23             + curr_1014.penyanyi_1014);
24         curr_1014 = curr_1014.next_1014;
25     }
26 }
27
28 // tampil playlist mundur
29 public void tampilMundur_1014() {
30     if (tail_1014 == null) {
31         System.out.println("Playlist kosong!");
32         return;
33     }
34     Lagu_2511531014 curr_1014 = tail_1014;
35     System.out.println("\n=== Playlist Mundur ===");
36     while (curr_1014 != null) {
37         System.out.println(
38             curr_1014.judul_1014
39             + " - "
40             + curr_1014.penyanyi_1014);
41         curr_1014 = curr_1014.prev_1014;
42     }
43 }
44

```

```

74
75 // cari lagu
76 public void carilagu_1014(String judulCari_1014) {
77     if (head_1014 == null) {
78         System.out.println("Playlist kosong!");
79         return;
80     }
81     Lagu_2511531014 curr_1014 = head_1014;
82     boolean ditemukan_1014 = false;
83     while (curr_1014 != null) {
84         if (curr_1014.judul_1014.equalsIgnoreCase(judulCari_1014)) {
85             System.out.println("Lagu ditemukan:");
86             System.out.println(
87                 curr_1014.judul_1014
88                 + " - "
89                 + curr_1014.penyanyi_1014);
90             ditemukan_1014 = true;
91             break;
92         }
93         curr_1014 = curr_1014.next_1014;
94     }
95     if (!ditemukan_1014) {
96         System.out.println("Lagu tidak ditemukan!");
97     }
98 }
99
100 // main program
101 public static void main(String[] args) {
102     Scanner input_1014 = new Scanner(System.in);
103     Musik_2511531014 playlist_1014 =
104         new Musik_2511531014();
105
106     int pilih_1014;
107     do {
108         System.out.println("\n=== Playlist Musik NIM: 2511531014 ===");
109         System.out.println("1. Tambah Lagu");
110         System.out.println("2. Hapus Lagu Pertama");
111         System.out.println("3. Lihat Playlist (Maju)");
112         System.out.println("4. Lihat Playlist (Mundur)");
113         System.out.println("5. Cari Lagu");
114         System.out.println("6. Keluar");
115         System.out.print("Pilihan: ");
116
117         pilih_1014 = input_1014.nextInt();
118         input_1014.nextLine();
119

```

```

120 switch (pilih_1014) {
121
122     case 1:
123         System.out.print("Judul Lagu: ");
124         String judul_1014 = input_1014.nextLine();
125
126         System.out.print("Penyanyi: ");
127         String penyanyi_1014 = input_1014.nextLine();
128
129         playlist_1014.tambahLagu_1014(judul_1014, penyanyi_1014);
130         break;
131
132     case 2:
133         playlist_1014.hapusLaguAwal_1014();
134         break;
135
136     case 3:
137         playlist_1014.tampilMaju_1014();
138         break;
139
140     case 4:
141         playlist_1014.tampilMundur_1014();
142         break;
143
144     case 5:
145         System.out.print("Masukkan judul lagu: ");
146         String cari_1014 = input_1014.nextLine();
147         playlist_1014.carilagu_1014(cari_1014);
148         break;
149
150     case 6:
151         System.out.println("Program selesai");
152         break;
153     default:
154         System.out.println("Pilihan tidak valid!");
155
156 }
157
158 } while (pilih_1014 != 6);
159 input_1014.close();
160 }
161

```

• Pembahasan

Program ini dibuat dengan mendefinisikan class Musik_2511531014 yang digunakan untuk mengelola playlist lagu menggunakan struktur data Double Linked List (DLL). Class ini berfungsi untuk menambahkan lagu, menghapus lagu, menampilkan playlist, dan mencari lagu dalam playlist.

1. Class ini memiliki atribut head_1014 dan tail_1014 yang digunakan sebagai penunjuk node pertama dan node terakhir pada Double Linked List. head digunakan untuk traversal maju, sedangkan tail digunakan untuk traversal mundur.

```

package TugasPekan6_2511531014;
import java.util.Scanner;
public class Musik_2511531014 {

    Lagu_2511531014 head_1014 = null;
    Lagu_2511531014 tail_1014 = null;

```

2. Method tambahLagu_1014() digunakan untuk menambahkan lagu baru di akhir playlist. Jika playlist masih kosong, maka node baru akan menjadi head dan tail. Jika playlist sudah berisi data, maka pointer next dari node terakhir diarahkan ke node baru, dan pointer prev node baru diarahkan ke node sebelumnya.

```
// tambah lagu di akhir
public void tambahLagu_1014(String judul_1014, String penyanyi_1014) {
    Lagu_2511531014 laguBaru_1014 =
        new Lagu_2511531014(judul_1014, penyanyi_1014);
    // jika playlist kosong
    if (head_1014 == null) {
        head_1014 = laguBaru_1014;
        tail_1014 = laguBaru_1014;
    } else {
        tail_1014.next_1014 = laguBaru_1014;
        laguBaru_1014.prev_1014 = tail_1014;
        tail_1014 = laguBaru_1014;
    }
    System.out.println("Lagu berhasil ditambahkan!");
}
}
```

3. Method hapusLaguAwal_1014() digunakan untuk menghapus lagu pertama pada playlist. Setelah head dipindahkan ke node berikutnya, pointer prev dari head baru diubah menjadi null. Jika playlist menjadi kosong, maka tail juga diubah menjadi null.

```
// hapus lagu pertama
public void hapusLaguAwal_1014() {
    if (head_1014 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    System.out.println("Lagu "
        + head_1014.judul_1014
        + " berhasil dihapus");
    head_1014 = head_1014.next_1014;
    if (head_1014 != null) {
        head_1014.prev_1014 = null;
    } else {
        tail_1014 = null;
    }
}
}
```

4. Method tampilMaju_1014() digunakan untuk menampilkan seluruh playlist dari awal ke akhir menggunakan traversal pointer next. Program akan menampilkan judul lagu dan nama penyanyi secara berurutan.

```
// tampil playlist maju
public void tampilMaju_1014() {
    if (head_1014 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    Lagu_2511531014 curr_1014 = head_1014;
    System.out.println("\n=== Playlist Maju ===");
    while (curr_1014 != null) {
        System.out.println(
            curr_1014.judul_1014
            + " - "
            + curr_1014.penyanyi_1014);
        curr_1014 = curr_1014.next_1014;
    }
}
}
```

5. Method tampilMundur_1014() digunakan untuk menampilkan playlist dari akhir ke awal menggunakan traversal pointer prev. Method ini menunjukkan keunggulan Double Linked List karena data dapat ditelusuri dua arah.

```
// tampil playlist mundur
public void tampilMundur_1014() {
    if (tail_1014 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    Lagu_2511531014 curr_1014 = tail_1014;
    System.out.println("\n== Playlist Mundur ==");
    while (curr_1014 != null) {
        System.out.println(
            curr_1014.judul_1014
            + " - "
            + curr_1014.penyanyi_1014);
        curr_1014 = curr_1014.prev_1014;
    }
}
```

6. Method `cariLagu_1014()` digunakan untuk mencari lagu berdasarkan judul lagu. Pencarian dilakukan menggunakan method `equalsIgnoreCase()` sehingga huruf besar dan kecil tidak memengaruhi hasil pencarian. Jika lagu ditemukan, program akan menampilkan data lagu tersebut.

```
// cari lagu
public void cariLagu_1014(String judulCari_1014) {
    if (head_1014 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    Lagu_2511531014 curr_1014 = head_1014;
    boolean ditemukan_1014 = false;
    while (curr_1014 != null) {
        if (curr_1014.judul_1014.equalsIgnoreCase(judulCari_1014)) {
            System.out.println("Lagu ditemukan:");
            System.out.println(
                curr_1014.judul_1014
                + " - "
                + curr_1014.penyanyi_1014);
            ditemukan_1014 = true;
            break;
        }
        curr_1014 = curr_1014.next_1014;
    }
    if (!ditemukan_1014) {
        System.out.println("Lagu tidak ditemukan!");
    }
}
```

7. Pada method `main()`, program menggunakan Scanner untuk menerima input dari pengguna. Program menampilkan menu utama yang berisi fitur tambah lagu, hapus lagu pertama, tampil playlist maju, tampil playlist mundur, cari lagu, dan keluar program. Setiap pilihan menu akan menjalankan method sesuai fungsi yang dipilih pengguna.

```

int pilih_1014;
do {
    System.out.println("\n== Playlist Musik NIM: 2511531014 ==");
    System.out.println("1. Tambah Lagu");
    System.out.println("2. Hapus Lagu Pertama");
    System.out.println("3. Lihat Playlist (Maju)");
    System.out.println("4. Lihat Playlist (Mundur)");
    System.out.println("5. Cari Lagu");
    System.out.println("6. Keluar");
    System.out.print("Pilihan: ");

    pilih_1014 = input_1014.nextInt();
    input_1014.nextLine();

    switch (pilih_1014) {

        case 1:
            System.out.print("Judul Lagu: ");
            String judul_1014 = input_1014.nextLine();

            System.out.print("Penyanyi: ");
            String penyanyi_1014 = input_1014.nextLine();

            playlist_1014.tambahLagu_1014(judul_1014, penyanyi_1014);
            break;

        case 2:
            playlist_1014.hapusLaguAwal_1014();
            break;

        case 3:
            playlist_1014.tampilMaju_1014();
            break;

```

```

        case 4:
            playlist_1014.tampilMundur_1014();
            break;

        case 5:
            System.out.print("Masukkan judul lagu: ");
            String cari_1014 = input_1014.nextLine();
            playlist_1014.cariLagu_1014(cari_1014);
            break;

        case 6:
            System.out.println("Program selesai");
            break;
        default:
            System.out.println("Pilihan tidak valid!");
    }

} while (pilih_1014 != 6);
input_1014.close();
}
}

```

8. Secara keseluruhan, program ini berhasil mengimplementasikan struktur data Double Linked List pada playlist musik. Setiap node saling terhubung dua arah menggunakan pointer next dan prev, sehingga data dapat dikelola secara dinamis dan traversal dapat dilakukan dari depan maupun belakang. Berikut Output yang dihasilkan

```
=== Playlist Musik NIM: 2511531014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu: Sailor Song
Penyanyi: Gigi Perez
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu Sailor Song berhasil dihapus

=== Playlist Musik NIM: 2511531014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
Playlist kosong!
```

```
=== Playlist Musik NIM: 2511531014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
Playlist kosong!

=== Playlist Musik NIM: 2511531014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: Sailor Song
Playlist kosong!

=== Playlist Musik NIM: 2511531014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Program selesai
```